

Using Self-Consistent Naive-Bayes to Detect Masquerades

Kwong H. Yung

Stanford University Statistics Department
390 Serra Mall; Stanford CA 94305-4020; USA
`khyung@stat.stanford.edu`

Abstract. To gain access to account privileges, an intruder masquerades as the proper account user. This paper proposes a new strategy for detecting masquerades in a multiuser system. To detect masquerading sessions, one profile of command usage is built from the sessions of the proper user, and a second profile is built from the sessions of the remaining known users. The sequence of the commands in the sessions is reduced to a histogram of commands, and the naive-Bayes classifier is used to decide the identity of new incoming sessions. The standard naive-Bayes classifier is extended to take advantage of information from new unidentified sessions. On the basis of the current profiles, a newly presented session is first assigned a probability of being a masquerading session, and then the profiles are updated to reflect the new session. As prescribed by the expectation-maximization algorithm, this procedure is iterated until both the probabilities and the profiles are self-consistent. Experiments on a standard artificial dataset demonstrate that this self-consistent naive-Bayes classifier beats the previous best-performing detector and reduces the missing-alarm rate by 40%.

Keywords: self-consistent naive-Bayes, expectation-maximization algorithm, semisupervised learning; masquerade detection, anomaly detection, intrusion detection.

1 Introduction

This paper presents a simple technique for identifying masquerading sessions in a multiuser system. Profiles of proper and intruding behavior are built on the sessions of known users. As new, unidentified sessions are presented, the profiles are adjusted to reflect the credibility of these new sessions. Based on the expectation-maximization algorithm, the proposed *self-consistent* naive-Bayes classifier markedly improves on earlier detection rates.

1.1 Motivation

Unauthorized users behave differently from proper users. To detect intruders, behaviors of proper users are recorded and deviant behaviors are flagged. Typically, each user account is assigned to a single proper user with a specified role.

Hence, future sessions under that user account can be compared against the recorded profile of the proper user. This approach to detecting masquerades is called anomaly detection.

More generally, intrusion detection is amenable to two complementary approaches, misuse detection and anomaly detection. Misuse detection trains on examples of illicit behavior; anomaly detection trains on examples of proper behavior. Because examples of intrusions are generally rare and novel attacks cannot be anticipated, anomaly detection is typically more flexible. In the context of masquerade detection, anomaly detection is especially appropriate because each user account is created with an assigned proper user, whose behavior must not deviate far from the designated role.

1.2 Previous Approaches

The artificial dataset used in this study has been studied by many researchers [3, 4, 7, 9, 11, 12]. Subsection 3.1 presents a summary of the dataset, and full details are available in [11]. Before this paper, the best result was achieved by using a naive-Bayes classifier with updating [9]. Yung [14] markedly improved detection rates but relied on confirmed updates. This paper also substantially improves over [9] and is competitive with [14].

There are at least two limitations to the updating scheme used in [9] and [11]. Principally, the detector must make a concrete decision whether to update the proper profile with the new session. In many cases, the judgment of a session is not always clear. Yet the detector is forced to make a binary decision before the detector can continue, because future decisions depend on the decision for the current case.

Furthermore, all future decisions depend on the past decisions, but the converse does not hold. In principle, the detector can defer the decision on the current session until additional new sessions from a later stage are studied. Yet in both [9] and [11], the detector is forced to make an immediate, greedy decision for each new session, without considering the information from future cases. Scores for previous sessions cannot be revised as new sessions are encountered. Hindsight cannot improve the detector's performance on earlier sessions, because backtracking is not allowed.

1.3 New Strategy

Without a concrete optimality criterion, there is no basis on which to judge the performance of the greedy sequential strategy. In general, however, the best greedy strategy is not always the best strategy overall. By ignoring strategies that involve backtracking, the greedy search may fail to make decisions most consistent with the entire dataset.

This paper presents an alternative strategy for identifying masquerading sessions. To start, an optimality criterion is defined by specifying a concrete objective function. Moreover, each new session is assigned a score indicating how likely that session is a masquerading session. Instead of a binary decision, the

detector assigns to the new session a probability between 0 and 1. This new session is then used to update the detector in a continuous fashion. The technique presented in this paper is an extension of the naive Bayes classifier used in [9] and will be called the *self-consistent naive-Bayes* classifier.

Although the self-consistent naive-Bayes classifier is a significant step beyond the simple naive-Bayes classifier used by Maxion and Townsend [9], three elements introduced there still apply here: the classification formulation, the bag-of-words model, and the naive-Bayes classifier. These three strategies are not unique to detecting masquerading sessions, but rather they appear in the far more general context of intrusion detection and text classification. Yung [14] provides a brief review of these three elements, within the context of identifying masquerading sessions.

2 Theory

The simple naive-Bayes classifier is typically built on a training set of labeled documents. Nigam and his colleagues [10] demonstrated that classification accuracy can be improved significantly by incorporating labeled as well as unlabeled documents in the training set. The algorithm for training this new *self-consistent* naive-Bayes classifier can be derived from the formalism of the EM-algorithm. Only the simplest version of this extended naive-Bayes classifier is presented here.

2.1 Likelihood from Test Sessions

Only two distinct classes of sessions are considered, the proper class and the masquerading class. The indicator *variable* $1_s = 1$ exactly when session s is a masquerading session. Let $1 - \epsilon$ and ϵ be the prior probabilities that a session is proper and masquerading, respectively. Moreover, let p_c and p'_c be the probability of command c in a proper and masquerading session, respectively.

The log-likelihood L_s of a test session s is simply, up to an additive constant

$$L_s = (1 - 1_s)(\log(1 - \epsilon) + \sum_{c=1}^C n_{sc} \log p_c) + 1_s(\log \epsilon + \sum_{c=1}^C n_{sc} \log p'_c), \quad (1)$$

where n_{sc} is the total count of command c in session s .

Assuming that all test sessions are generated independently of each other, the cumulative log-likelihood L^t after t test sessions is, up to an additive constant

$$L_+^t = \sum_{s=1}^t L_s = w_+^t \log(1 - \epsilon) + \sum_{c=1}^C n_{+c}^t \log p_c + w_+^{tt} \log \epsilon + \sum_{c=1}^C n_{+c}^{tt} \log p'_c, \quad (2)$$

where

$$w_+^t = \sum_{s=1}^t (1 - 1_s), w_+^{tt} = \sum_{s=1}^t 1_s, n_{+c}^t = \sum_{s=1}^t (1 - 1_s) n_{sc}, n_{+c}^{tt} = \sum_{s=1}^t 1_s n_{sc}. \quad (3)$$

Here w_+^t and $w_+^{t'} = t - w_+^t$ are the cumulative numbers of proper and masquerading sessions, respectively; n_{+c}^t and $n_{+c}^{t'}$ are the cumulative counts of command c amongst proper sessions and masquerading sessions, respectively, in the t total observed test sessions.

2.2 Likelihood from Training Sessions

Now let n_{+c}^0 denote the total count of command c among proper sessions in the training set. Likewise, let $n_{+c}^{'0}$ denote the total count of command c among masquerading sessions in the training set. Letting r denote a session in the training set,

$$n_{+c}^0 = \sum_r (1 - 1_r) n_{rc}, n_{+c}^{'0} = \sum_r 1_r n_{rc}. \quad (4)$$

Assuming that the sessions in the training set are generated independently, the log-likelihood L_+^0 of the proper and masquerading sessions in the training set is

$$L_+^0 = \sum_{c=1}^C n_{+c}^0 \log p_c + \sum_{c=1}^C n_{+c}^{'0} \log p'_c. \quad (5)$$

This log-likelihood L_+^0 is useful for providing initial estimates of p_c and p'_c but provides no information about ϵ .

2.3 Posterior Likelihood

Rare classes and rare commands may not be properly reflected in the training set. To avoid zero estimates, smoothing is typically applied to the maximum-likelihood estimators. This smoothing can also be motivated by shrinkage estimation under Dirichlet priors on the parameters ϵ , p_c , and p'_c .

Here the parameters ϵ , p_c , and p'_c are drawn from known prior distributions with fixed parameters, and a simple standard Bayesian analysis is applied. Suppose that

$$(1 - \epsilon, \epsilon) \sim \text{Beta}(\beta, \beta'), \quad (6)$$

$$p \sim \text{Dirichlet}(\alpha), \quad (7)$$

$$p' \sim \text{Dirichlet}(\alpha'), \quad (8)$$

where α , α' , β , and β' are taken to be known fixed constants specified in advance. Then the cumulative posterior log-likelihood $\tilde{L}^t$ is, up to an additive constant

$$\begin{aligned} \tilde{L}^t = & (\beta - 1 + w_+^t) \log(1 - \epsilon) + \sum_{c=1}^C (\alpha_c - 1 + n_{+c}^0 + n_{+c}^t) \log p_c + \\ & (\beta' - 1 + w_+^{t'}) \log \epsilon + \sum_{c=1}^C (\alpha'_c - 1 + n_{+c}^{'0} + n_{+c}^{t'}) \log p'_c. \end{aligned} \quad (9)$$

Here w_+^t , $w_+^{t'}$, n_{+c}^t , and $n_{+c}^{t'}$, defined in Equation 3, are cumulative quantities determined by the t available test sessions; n_{+c}^0 and $n_{+c}^{'0}$, defined in Equation 4, are the fixed quantities determined by the training sessions.

2.4 Shrinkage Estimators

Equation 9 gives the cumulative posterior log-likelihood $\tilde{L}^t$ after observing t test sessions, in addition to the training sessions. Shrinkage estimators are just the maximum-likelihood estimators calculated from the posterior log-likelihood $\tilde{L}^t$. So cumulative shrinkage estimators $\hat{\epsilon}^t$, $\hat{p}_c^t$ and $\hat{p}_c'^t$ for ϵ , p_c and p_c' after $t > 0$ sessions are

$$\hat{\epsilon}^t = \frac{\beta' - 1 + w_+^t}{\beta - 1 + \beta' - 1 + t}, \quad (10)$$

$$\hat{p}_c^t = \frac{\alpha_c - 1 + n_{+c}^0 + n_{+c}^t}{\sum_{v=1}^C (\alpha_v - 1 + n_{+v}^0 + n_{+v}^t)}, \quad (11)$$

$$\hat{p}_c'^t = \frac{\alpha'_c - 1 + n_{+c}^{\prime 0} + n_{+c}^{\prime t}}{\sum_{v=1}^C (\alpha'_v - 1 + n_{+v}^{\prime 0} + n_{+v}^{\prime t})}. \quad (12)$$

2.5 Complete Log-Likelihood

Initially, the training set is used to build the naive-Bayes classifier. As a new unidentified session is presented to the classifier, that new session is scored by the classifier and used to update the classifier. This scoring and updating procedure is repeated until convergence. As each new session is presented, the classifier is updated in a self-consistent manner.

For a session s in the training set, the identity is known. Specifically, $1_s = 0$ for a proper session, and $1_s = 1$ for a masquerading session in the training set. For a new unidentified session, 1_s is a missing variable that must be estimated from the available data.

2.6 EM-Algorithm for Naive-Bayes Model

The EM-algorithm [1] is an iterative procedure used to calculate the maximum-likelihood estimator from the complete log-likelihood. Each iteration of the EM-algorithm includes two steps, the expectation step and the maximization step.

In the expectation step, the indicators 1_s are replaced by their expectations, calculated from the current estimates of model parameters. In the maximization step, the new estimates of the model parameters are calculated by maximizing the expected log-likelihood. For each stage t , these two-steps are repeated through multiple iterations until convergence. Below the iterations of the EM-algorithm during a fixed stage t are described.

Let $\theta^t = (\epsilon^t, p_c^t, p_c'^t)$ denote the estimate of $\theta = (\epsilon, p_c, p_c')$ at stage t , after observing the first t test sessions. For each known session r in the training set, 1_r is a known constant. So n_{+c}^0 and $n_{+c}^{\prime 0}$ of Equation 4 remain fixed even as θ^t changes.

For each *unidentified* test session $s = 1, 2, \dots, t$, the expectation step estimates $E[1_s | c_s; \theta^t] = P(1_s = 1 | c_s; \theta^t)$ via Bayes rule as

$$P(1_s = 1 | c_s; \theta^t) = \frac{P(1_s = 1; \theta^t)P(c_s | 1_s = 1; \theta^t)}{P(c_s; \theta^t)}, \quad (13)$$

where

$$P(c_s; \theta^t) = P(1_s = 0; \theta^t)P(c_s | 1_s = 0; \theta^t) + P(1_s = 1; \theta^t)P(c_s | 1_s = 1; \theta^t). \quad (14)$$

For the expectation step of the current iteration, the estimate $\hat{\theta}^t$ from the maximization step of the previous iteration is used for θ^t .

The maximization step calculates updated estimate $\hat{\theta}^t$ of the parameter θ^t . The usual cumulative shrinkage estimators of Equations 10–12 are used. However, the counts w_+^t , $w'_+{}^t$, n_{+c}^t , and $n'_{+c}{}^t$ are now no longer known constants but rather are random variables. These random variables are replaced by their estimates from the expectation step above. In other words, the maximization step of the current iteration uses the estimates from the expectation step of the current iteration.

3 Results

The technique proposed in this paper is tested on a standard dataset previously analyzed by other authors using different techniques. The dataset contains command logs from 50 users on a UNIX multiuser system. To create artificial intrusion sessions, sessions from another 20 users are injected at random into the command logs of the 50 known users. The dataset contains a training portion, without masquerading sessions. The remaining test portion contains possible masquerading sessions from the artificially created intruders. A summary of the dataset is provided below, but full details are provided in [11].

3.1 Reference Dataset

For privacy reasons, the command logs were stripped of the command options and arguments, leaving only the commands themselves. For each of 70 users, the first 15000 recorded commands were kept. Then 50 users were chosen at random to serve as the dataset, and the remaining 20 users were excluded, except to provide artificial intrusion data. For each user, the sequence of 15000 commands was divided into 150 sequential blocks, each containing 100 commands in sequence. Each artificially created block is treated as a session. Therefore, each of the 50 users had 150 sessions of 100 commands.

The first 50 sessions of the 50 users are assumed to be free of intruders. These 50 sessions for 50 users constitute the training set. The remaining 100 sessions for each of the 50 users form the test set. Sessions from the 20 excluded users are injected into the test session of the 50 known users. The extra sessions in the test sets are then removed, leaving just 100 test sessions for each of the 50 known users.

To prevent over-training on the data creation process, the exact details of how the test set is created cannot be used in the intrusion detection procedure. Essentially, once an intruder has broken into a user account, he is likely to return

on the very next session. A proper session is followed by an intruder session with 0.01 probability. A masquerading session is followed by a masquerading session from the same intruder with 0.80 probability. A chain of masquerading sessions is taken from the intruder in a contiguous sequence. Out of the total 5000 test sessions, there were 231 total masquerading sessions, and the remaining 4769 were proper sessions. These aspects of the data creation are not used as part of the classification procedure.

3.2 Experimental Design

The details for one single experiment are described below. The example shows how masquerading sessions on User 12 were detected. For other users, the same analysis applies.

The full training set was divided into two classes. The proper class consisted of 50 sessions for User 12. The masquerade class consisted of the 50 sessions for each of the other 49 users. So the masquerade class was built on a total of 2450 sessions.

The test set for user 12 had 100 sessions, all purportedly proper sessions. The classifier for user 12 was then run against each of the 100 sessions in a sequential fashion. As a new session is added, the scores of the previous sessions are also updated. At the end of the 100 sessions, the final set of all 100 scores were recorded.

The ROC curve for user 12 was generated by thresholding against the *final* set of scores, after all test sessions were presented sequentially. In this case, a session's score is the probability that the session was generated from the proper profile. Therefore, sessions with lower scores were flagged before sessions with higher scores.

3.3 Classifier Settings

A self-consistent naive-Bayes classifier was built for each individual user. The algorithm is outlined in Subsection 2.6. The initial naive-Bayes was built on the training set. As each new session is presented, that new session is first scored by the previous classifier. Afterwards, all the unidentified sessions are used to construct the self-consistent naive-Bayes. This procedure is repeated for each new session.

In the full self-consistent naive-Bayes classifier presented in Section 2, the fraction ϵ of masquerading sessions was adjusted in each iteration. In the results presented here, the fraction $\epsilon = 0.50$ was held fixed and was not estimated as part of the EM-algorithm. Keeping the prior weight $(1 - \epsilon, \epsilon)$ fixed at $(0.50, 0.50)$ allowed for faster convergence and did not adversely affect the final results.

For each new session, the EM-algorithm was used to adjust the fractions p_c and p'_c . The Dirichlet parameters $\alpha_c = 1.01$ and $\alpha'_c = 1.01$ for all $c \in \mathbf{C}$ were chosen to match the parameters used in [9]. The EM-algorithm was iterated until the change between iteration i and $i + 1$, measured by the quantity $\|p^{(i+1)} -$

$p^{(i)}||^2 + ||p^{(i+1)} - p^{(i)}||^2$ averaged over the total number of estimated parameters, was less than some tolerance, set to be 2.2×10^{-16} . In practice, the algorithm was not sensitive to the precise tolerance.

3.4 Composite ROC Curves

A separate classifier was created for each user and used to score that user’s test sessions. The scores of test sessions from one user can be compared because those scores were assigned by that user’s classifier. However, scores from different users cannot easily be compared because those scores are assigned from different classifiers.

To evaluate a strategy’s success over the entire dataset of 50 users, a composite ROC curve can be useful. There are several common ways to integrate the individual 50 ROC curves into one single curve, but all methods are at best arbitrary.

In this paper, the scores from the 50 different classifiers were taken at face value. The 5000 total test sessions from the 50 user were sorted, and sessions with the lowest scores were flagged first. Because the scores were probability values, this global thresholding strategy has some merit. This method for constructing the composite ROC curve was also used in [9] and [11]. Because the same methodology was used in these previous papers, the composite ROC curves can be meaningfully compared.

3.5 Experimental Results

Two ROC curves are used to compare the self-consistent naive-Bayes classifier to the one-step discrete adaptive naive-Bayes classifier of [9]. These curves together demonstrate that the self-consistent naive-Bayes classifier offers significant improvements. All results reported here are based on offline evaluation, after all the sessions have been presented sequentially. Online evaluation and deferral policies are discussed in a separate paper.

Figure 1 shows in fine detail the composite ROC curves of all 50 users, for false-alarm rate 0.00–0.10. Indeed, the self-consistent naive-Bayes outperforms the adaptive naive-Bayes uniformly for all but the small portion of false-alarm rate 0.00–0.01. In particular, for false-alarm rate 0.013, the self-consistent naive-Bayes reduces the missing-alarm rate of the adaptive naive-Bayes by 40%.

Figure 2 shows the ROC curve for user 12 alone. As noted by the other authors [11, 9], user 12 is a challenging case because the masquerading sessions in the test set appear similar to the proper sessions. On user 12, the adaptive naive-Bayes performs worse than random guessing, but self-consistent naive-Bayes encounters little difficulty. In fact, the 50 ROC curves over each individual user show that self-consistent naive-Bayes typically outperforms adaptive naive-Bayes.

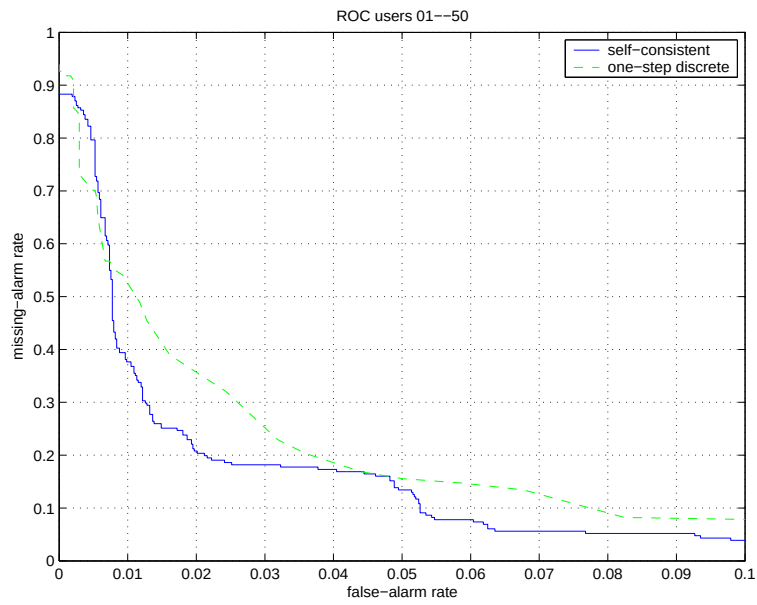


Fig. 1. ROC curve over users 01-50, less than 0.10 false-alarm rate.

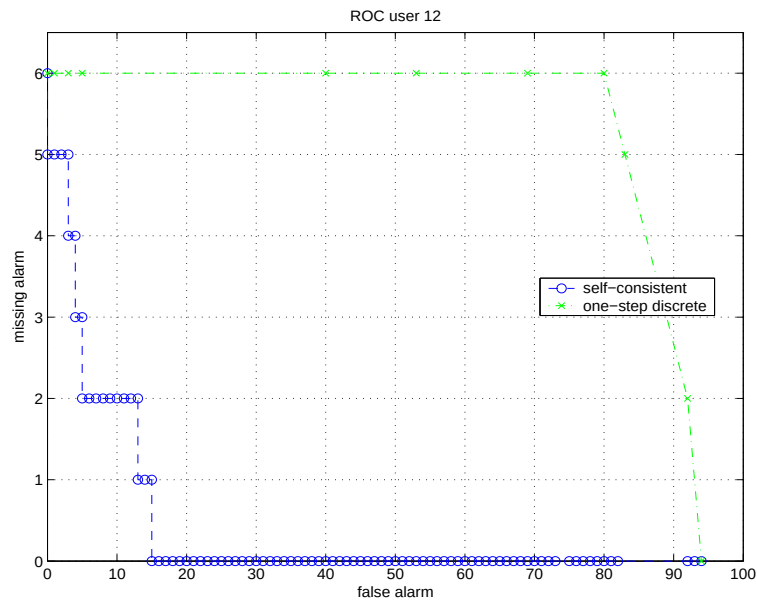


Fig. 2. ROC curve user 12

4 Discussion

The self-consistent naive-Bayes classifier outperforms the adaptive naive-Bayes classifier in all practical circumstances, where only low false-alarm rates can be tolerated. For false-alarm rate 0.013, the self-consistent naive-Bayes classifier lowers the missing-alarm rate by 40%.

4.1 Long-Term Scenario

Unlike the adaptive naive-Bayes, the self-consistent naive-Bayes is not forced to make a binary decision as each new session is presented. This advantage will become more dramatic as the number of users and the number of sessions increase. As the classifier learns from more cases, mistakes from earlier decisions are propagated onwards and magnified. Because the adaptive naive-Bayes is forced to make a binary decision immediately, it is also more likely to make errors. In a long-term scenario, the improvements of the self-consistent naive-Bayes classifier are expected to become far more pronounced.

4.2 Computation Time

Although the self-consistent naive-Bayes classifier offers a great number of advantages, more computation is required. The iterative procedure used to calculate the probabilities and to update the profile must be run until convergence. This additional effort allows the self-consistent naive-Bayes classifier to assign scores in a non-greedy fashion. Naturally, searching through the space of models requires more effort than the simple greedy approach.

In practice, convergence is often quick because most sessions have probability scores at the extremes, near 0 or 1. In the context of sequential presentation, only one single session is presented at a time. Computing the score of a single session requires little additional effort. Moreover, the set of scores from one stage can be used as the starting point to calculate scores for the next stage. This incremental updating approach relieves the potential computational burden.

4.3 Nonstationary User Behavior

For some users, the sessions in the test set differ significantly from the sessions in the training set. Because user behavior changes unpredictably with time, a detector based on the old behavior uncovered in the training set can raise false alarms. This high rate of false alarms can render any detector impractical, because there are not enough resources to investigate these cases.

In certain real world applications, however, identifying changed behavior may well be useful because a proper user can misuse his own account privileges for deviant and illicit purposes [8]. If the detector is asked to detect all large deviations in behavior, then flagging unusual sessions from the proper user is acceptable. This redefinition is especially useful to prevent a user from abusing his own privileges.

4.4 Future Directions

Only the simplest model of user sessions has been presented. In [10], EM-algorithm was applied to more elaborate and realistic models. These extensions offer much potential, and future work will explore these possibilities.

The iterative nature of the EM-algorithm is quite intuitive. In fact, many instantiations [2, 5, 6, 13] of the EM-algorithm existed well before the general EM-algorithm was formulated in [1]. Moreover, the self-consistency paradigm can be applied even to classifiers not based on a likelihood model. A self-consistent version can be constructed for potentially any classifier, in the same way that a classifier can be updated by including the tested instance as part of the modified training set.

The naive-Bayes classifier relies only on command frequencies. As a user's behavior changes over time, the profile built from past sessions become outdated. An exponential-weighting process can be applied to counts from past sessions. For the naive-Bayes classifier, this reweighting of counts is especially simple. Such an extension becomes even more valuable in realistic scenarios, where a sequential classifier is used in an online context for a long time.

5 Summary

In previous approaches to detecting masquerading sessions, the detector was forced to make a binary decision about the identity of a new session. The self-consistent naive-Bayes does not make a binary decision but rather estimates the probability of a session being a masquerading session. Moreover, past decisions can be adjusted to accommodate newer sessions. Experiments prove that this sensible extension markedly improves over more restrictive adaptive approaches, by reducing the missing-alarm rate by 40%.

The self-consistent naive-Bayes classifier extends the usual naive-Bayes classifier by taking advantage of information from unlabeled instances. New instance are assigned probabilistic labels, and then the profiles are updated in accordance with the assigned probabilities of the new instances. This procedure is iterated until convergence to a final set of probabilities which are consistent with the updated profile.

By its very nature, the self-consistent naive-Bayes classifier is adaptive to the new sessions. Moreover, information from new sessions is also used to adjust scores of previous new sessions. In this way, the scores of sessions are assigned in a self-consistent manner. As a specific instance of the EM-algorithm, the self-consistent naive-Bayes classifier finds a model optimal with respect to its likelihood given the available data.

6 Acknowledgments

This research project was funded in part by the US Department of Justice grant 2000-DT-CX-K001. Jerome H. Friedman of the Stanford University Statistics

Department provided invaluable advice through many helpful discussions. Jeffrey D. Ullman of the Stanford University Computer Science Department introduced the author to the field of intrusion detection and offered insightful critiques throughout the past three years. The author is grateful for their guidance and support.

References

1. Arthur P. Dempster, Nam M. Laird, and Donald B. Rubin. "Maximum likelihood from incomplete data via the EM algorithm." *Journal of the Royal Statistical Society, Series B*, 39(1), 1–38, 1977.
2. N. E. Day. "Estimating the components of a mixture of two normal distributions." *Biometrika*, **56**, 463–474, 1969.
3. William DuMouchel. "Computer intrusion detection based on Bayes factors for comparing command transition probabilities." Technical Report 91, National Institute of Statistical Sciences, Research Triangle Park, North Carolina 27709-4006, 1999.
4. William DuMouchel and Matthias Schonlau. "A comparison of test statistics for computer intrusion detection based on principal components regression of transition probabilities." *Proceedings of the 30th Symposium on the Interface: Computing Science and Statistics*, 30:404–413, 1999.
5. Victor Hasselblad. "Estimation of parameters for a mixture of normal distributions." *Technometrics*, **8**, 431–444, 1966.
6. Victor Hasselblad. "Estimation of finite mixtures of distributions from the exponential family." *Journal of American Statistical Association*, **64**, 1459–1471, 1969.
7. Wen-Hua Ju and Yehuda Vardi. "A hybrid high-order Markov chain model for computer intrusion detection." Technical Report 92, National Institute for Statistical Sciences, Research Triangle Park, North Carolina 27709-4006, 1999.
8. Vernon Loeb. "Spy case prompts computer search." *Washington Post*, 05 March 2001, page A01.
9. Roy A. Maxion and Tahlia N. Townsend. "Masquerade Detection Using Truncated Command Lines." *International Conference on Dependable Systems and Networks (DSN-02)*, pp. 219–228, Washington, DC, 23–26 June 2002. IEEE Computer Society Press, Los Alamitos, California, 2002.
10. Kamal Nigam, Andrew K. McCallum, Sebastian Thrun, Tom Mitchell. "Text classification from labeled and unlabeled documents using EM." *Machine Learning*, 39:2:103–134, 2000.
11. Matthias Schonlau, William DuMouchel, Wen-Hua Ju, Alan F. Karr, Martin Theus, and Yehuda Vardi. "Computer intrusion: detecting masquerades." *Statistical Science*, 16(1):58–74, February 2001.
12. Matthias Schonlau and Martin Theus. "Detecting masquerades in intrusion detection based on unpopular commands." *Information Processing Letters*, 76(1–2):33–38, November 2000.
13. John H. Wolfe. "Pattern clustering by multivariate mixture analysis." *Multivariate Behavioral Research*, **5**, 329–350, 1970.
14. Kwong H. Yung. "Using feedback to improve masquerade detection." *Lecture Notes in Computer Science: Applied Cryptography and Network Security*. Proceedings of the ACNS 2003 Conference. LNCS **2846**, 48–62. Springer-Verlag, October 2003.